

Week 2 - Assignment

SECTION – A

1. Display all columns and rows from customers table

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL query: `SELECT * FROM customers;`. The 'Results' tab shows the output of the query, which is a table with 10 columns: `customer_id`, `first_name`, `last_name`, `email`, `city`, `state`, `join_date`, and `is_premium`. The table contains 10 rows of data. The 'Output' tab shows the execution details: `1 16:47:13 SELECT * FROM customers LIMIT 0, 1000`, with a message '8 row(s) returned' and a duration of '0.000 sec / 0.000 sec'.

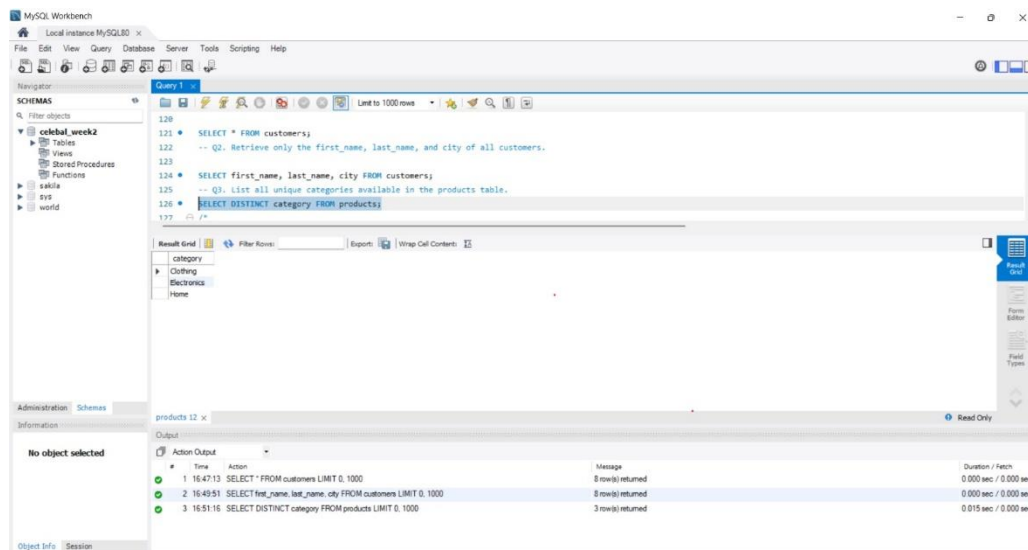
customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@gmail.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@gmail.com	Ahmedabad	Gujarat	2024-03-20	0
103	Rohan	Gupta	rohan.g@gmail.com	Delhi	Delhi	2024-03-30	1
104	Sneha	Roddy	sneha.r@gmail.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@gmail.com	Japur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@gmail.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehra	karan.m@gmail.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@gmail.com	Kochi	Kerala	2024-08-30	0

2. Retrieve first_name, last_name and city

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL query: `SELECT first_name, last_name, city FROM customers;`. The 'Results' tab shows the output of the query, which is a table with 3 columns: `first_name`, `last_name`, and `city`. The table contains 10 rows of data. The 'Output' tab shows the execution details: `1 16:47:13 SELECT * FROM customers LIMIT 0, 1000` and `2 16:49:51 SELECT first_name, last_name, city FROM customers LIMIT 0, 1000`, with a message '8 row(s) returned' and a duration of '0.000 sec / 0.000 sec'.

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Roddy	Hyderabad
Vikram	Singh	Japur
Ananya	Iyer	Chennai
Karan	Mehra	Pune
Divya	Nair	Kochi

3. List unique categories in products table



4. Identify the Primary Keys of each table. Why Primary Key must be UNIQUE and NOT NULL?

Answer:

customers -> customer_id

products -> product_id

orders -> order_id

order_items -> item_id

Why UNIQUE?

1. Prevents duplicate records.
2. Uniquely identifies each row.
3. Maintains data integrity.

Why NOT NULL?

1. Every record must have an identifier.
2. NULL cannot uniquely identify a row.
3. Required for establishing relationships.

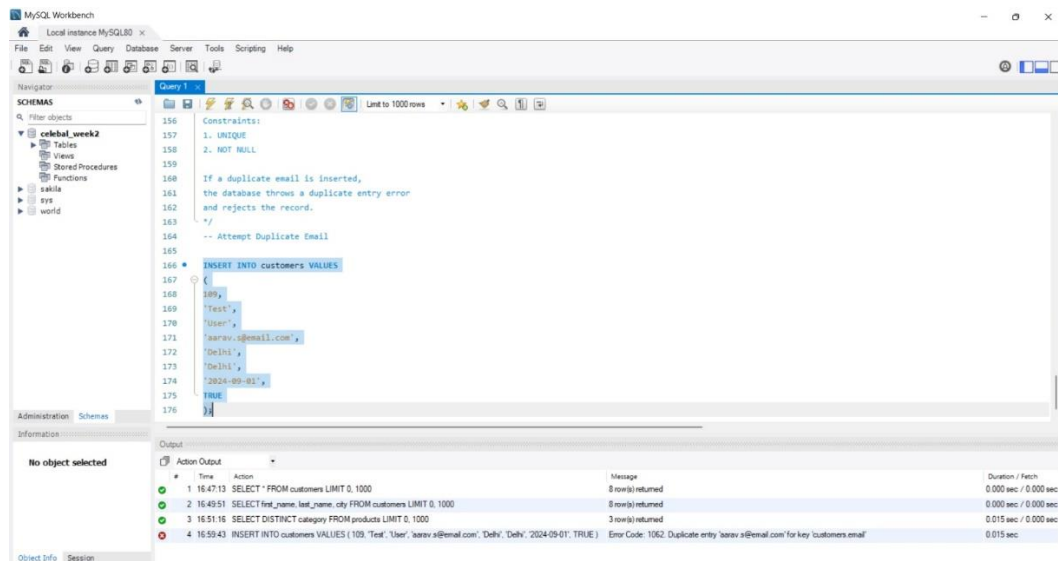
**5. What constraints are applied to the email column in the customers table?
What would happen if you tried to insert a duplicate email?**

Answer:

Constraints:

1. UNIQUE
2. NOT NULL

If a duplicate email is inserted,
the database throws a duplicate entry error
and rejects the record.



The screenshot shows the MySQL Workbench interface. The main editor window displays the following SQL code:

```
156 Constraints:
157 1. UNIQUE
158 2. NOT NULL
159
160 If a duplicate email is inserted,
161 the database throws a duplicate entry error
162 and rejects the record.
163 */
164 -- Attempt Duplicate Email
165
166 INSERT INTO customers VALUES
167 (
168 109,
169 'Test',
170 'User',
171 'saevas@gmail.com',
172 'Delhi',
173 '16-11-11',
174 '2024-09-01',
175 TRUE
176 )
```

The output window at the bottom shows the execution results of the queries. The final query (Query 4) resulted in an error:

#	Time	Action	Message	Duration / Fetch
1	16:47:13	SELECT * FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
2	16:49:51	SELECT first_name, last_name, city FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
3	16:51:16	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
4	16:59:43	INSERT INTO customers VALUES (109, 'Test', 'User', 'saevas@gmail.com', 'Delhi', '2024-09-01', TRUE)	Error Code: 1062 Duplicate entry 'saevas@gmail.com' for key 'customers.email'	0.015 sec

6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

```
-- INSERT INTO products VALUES ( 209, 'Test Product', 'Electronics', 'TestBrand', -50,
100 );

/*
```

Expected Result:

The insertion fails.

Reason:

CHECK (unit_price > 0)

the CHECK constraint prevents negative values
from being inserted into unit_price.

*/

The screenshot shows the MySQL Workbench interface. The main editor displays a SQL query that attempts to insert a product with a negative unit price. The query is as follows:

```
-- Try inserting a product with unit_price = -50.
-- What happens and which constraint prevents it?
/*
-- INSERT INTO products VALUES ( 209, 'Test Product', 'Electronics', 'TestBrand', -50, 100 );
*/

-- Expected results:
-- The insertion fails.
-- Reason:
-- CHECK (unit_price > 0)
-- The CHECK constraint prevents negative values
-- from being inserted into unit_price.
--
-- OK, test CHECK constraint
--
-- INSERT INTO products VALUES
-- (
-- 209,
-- 'Test Product',
-- 'Electronics',
-- 'TestBrand',
-- -50,
-- 100
-- );
```

The Output pane at the bottom shows the execution results of the query. The first three statements executed successfully, returning 8 rows each. The fourth statement, which was a duplicate entry error, also returned 3 rows. The fifth statement, the INSERT statement, failed with the following error message:

#	Time	Action	Message	Duration / Fetch
1	16:47:13	SELECT * FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
2	16:49:51	SELECT first_name, last_name, city FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
3	16:51:16	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
4	16:59:43	INSERT INTO customers VALUES (109, 'Test', 'User', 'aasav.s@email.com', 'Delhi', 'Delhi', '2024-09-01', TRUE)	Error Code: 1062. Duplicate entry 'aasav.s@email.com' for key 'customers_email'	0.015 sec
5	17:03:55	INSERT INTO products VALUES (209, 'Invalid Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3619. Check constraint 'products_chk_1' is violated.	0.000 sec

SECTION – B

7. Retrieve all orders with status = 'Delivered'.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
-- SECTION B - Filtering & Optimization (WHERE, Indexes)
-- Q7. Retrieve all orders with status = 'Delivered'.
SELECT *
FROM orders
WHERE status = 'Delivered';
```

The Result Grid displays the following data:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1898.00

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
2	16:49:51	SELECT first_name, last_name, city FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
3	16:51:16	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
4	16:59:43	INSERT INTO customers VALUES (109, 'Test', 'User', 'aarav.a@email.com', 'Delhi', 'Delhi', '2024-09-01', TRUE)	Error Code: 1062. Duplicate entry 'aarav.a@email.com' for key 'customers_email'	0.015 sec
5	17:03:55	INSERT INTO products VALUES (209, 'Invalid Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
6	18:04:57	SELECT * FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	6 row(s) returned	0.125 sec / 0.000 sec

8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
FROM orders
WHERE status = 'Delivered';

-- Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.
SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
```

The Result Grid displays the following data:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
3	16:51:16	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
4	16:59:43	INSERT INTO customers VALUES (109, 'Test', 'User', 'aarav.a@email.com', 'Delhi', 'Delhi', '2024-09-01', TRUE)	Error Code: 1062. Duplicate entry 'aarav.a@email.com' for key 'customers_email'	0.015 sec
5	17:03:55	INSERT INTO products VALUES (209, 'Invalid Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
6	18:04:57	SELECT * FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	6 row(s) returned	0.125 sec / 0.000 sec
7	18:08:05	SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
218 WHERE category = 'Electronics'
219 AND unit_price > 2000;
220
221 -- Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.
222 SELECT *
223 FROM customers
224 WHERE state = 'Maharashtra'
225 AND YEAR(join_date) = 2024;
```

The Result Grid shows the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehra	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
4	16:59:43	INSERT INTO customers VALUES (109, 'Test', 'User', 'aarav.s@email.com', 'Delhi', '2024-09-01', TRU...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	0.015 sec
5	17:03:55	INSERT INTO products VALUES (209, 'Invalid Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
6	18:04:57	SELECT * FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	6 row(s) returned	0.125 sec / 0.000 sec
7	18:08:05	SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
8	18:09:44	SELECT * FROM customers WHERE state = 'Maharashtra' AND YEAR(join_date) = 2024 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
224 WHERE state = 'Maharashtra'
225 AND YEAR(join_date) = 2024;
226
227 -- Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.
228 SELECT *
229 FROM orders
230 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
231 AND status <> 'Cancelled';
```

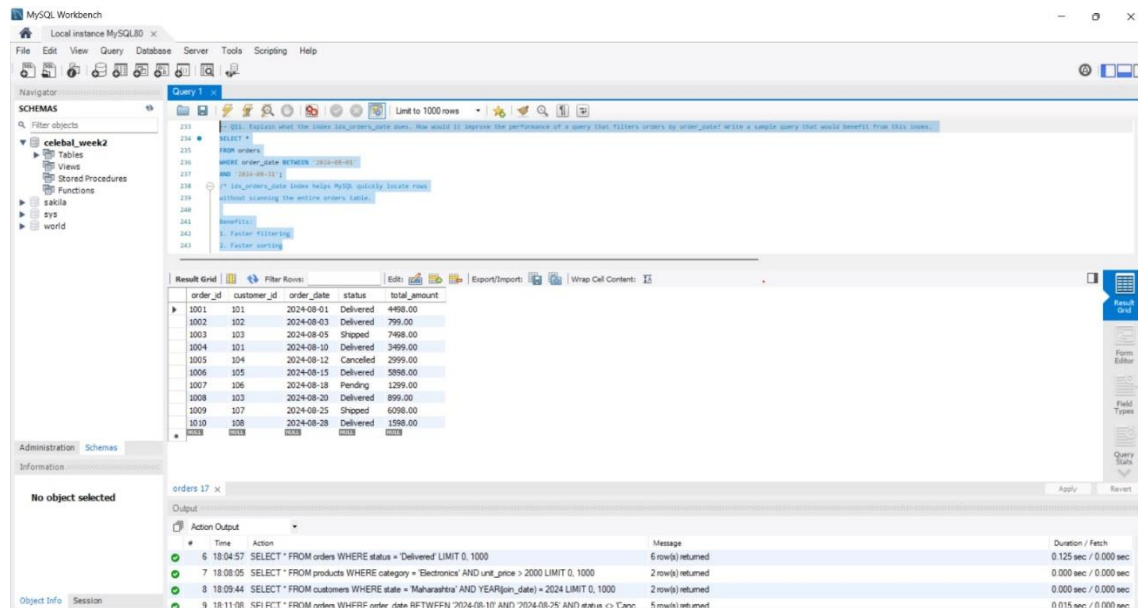
The Result Grid shows the following data:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5998.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
5	17:03:55	INSERT INTO products VALUES (209, 'Invalid Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
6	18:04:57	SELECT * FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	6 row(s) returned	0.125 sec / 0.000 sec
7	18:08:05	SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
8	18:09:44	SELECT * FROM customers WHERE state = 'Maharashtra' AND YEAR(join_date) = 2024 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
9	18:11:08	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Canc...	5 row(s) returned	0.015 sec / 0.000 sec

11.Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.



The screenshot shows MySQL Workbench with a query window. The query is as follows:

```

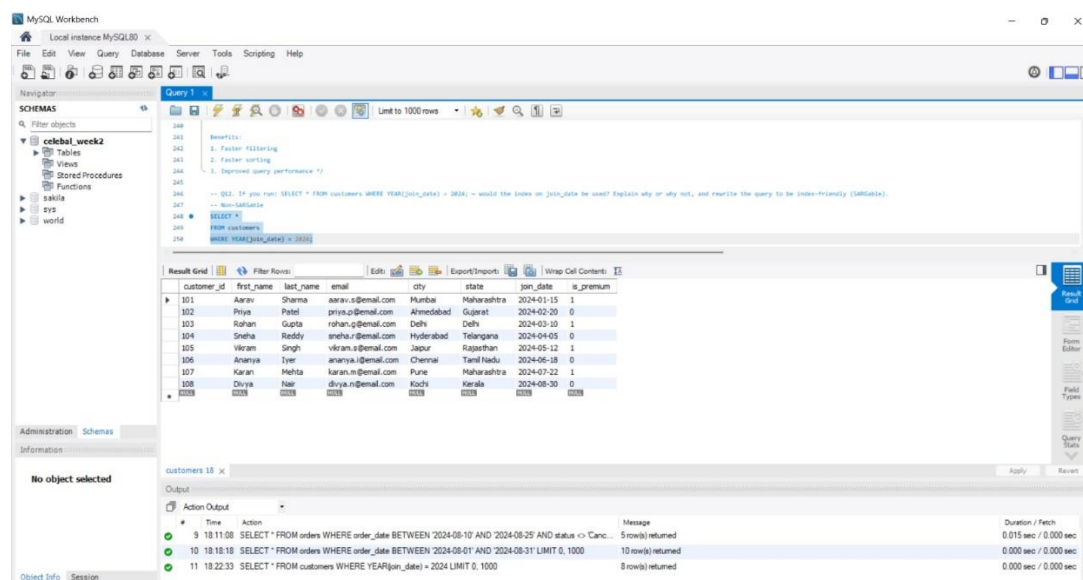
-- Q11: Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date with a sample query that would benefit from this index.
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-01'
AND '2024-08-10';
-- idx_orders_date index helps MySQL quickly locate rows
-- without scanning the entire orders table.
-- Faster Filtering
-- Faster sorting

```

The results window shows 17 rows of order data. Below the results, the 'Action Output' pane shows the execution plan for the query:

#	Time	Action	Message	Duration / Fetch
6	18.04.57	SELECT * FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	6 row(s) returned	0.125 sec / 0.000 sec
7	18.08.05	SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
8	18.09.44	SELECT * FROM customers WHERE state = 'Maharashtra' AND YEAR(join_date) = 2024 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
9	18.11.08	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-10' AND status <> 'Cancelled'	5 row(s) returned	0.015 sec / 0.000 sec

12.If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).



The screenshot shows MySQL Workbench with a query window. The query is as follows:

```

-- Q12: If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).
-- Non-SARGable
SELECT *
FROM customers
WHERE YEAR(join_date) = 2024;

```

The results window shows 18 rows of customer data. Below the results, the 'Action Output' pane shows the execution plan for the query:

#	Time	Action	Message	Duration / Fetch
9	18.11.08	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-10' AND status <> 'Cancelled'	5 row(s) returned	0.015 sec / 0.000 sec
10	18.16.18	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-10' LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
11	18.22.33	SELECT * FROM customers WHERE YEAR(join_date) = 2024 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

celebrat_week2

Tables

Views

Stored Procedures

Functions

sys

world

Query 1

Limit to 1000 rows

```
-- Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024, - would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARable).
-- Non-SARable
SELECT *
FROM customers
WHERE YEAR(join_date) = 2024;
-- SARable version
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
AND join_date < '2025-01-01';
```

Result Grid

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@gmail.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Fehel	priya.p@gmail.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@gmail.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@gmail.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@gmail.com	Japur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@gmail.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehra	karan.m@gmail.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@gmail.com	Kochi	Kerala	2024-08-30	0

customers 19 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
10	18:18:18	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31' LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
11	18:22:33	SELECT * FROM customers WHERE YEAR(join_date) = 2024 LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
12	18:25:05	SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01' LIMIT 0, 1000	8 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

SECTION – C

13. Count the total number of orders in the orders table.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
210 WHERE YEAR(join_date) = 2024;
211 -- Table Version
212 SELECT *
213 FROM customers
214 WHERE join_date >= '2020-01-01'
215 AND join_date < '2025-01-01';
216
217 -- Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
218 -- Q13. Count the total number of orders in the orders table.
219 SELECT COUNT(*) AS total_orders
220 FROM orders;
```

The Result Grid shows the output of the query:

total_orders
10

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
11	18:22:33	SELECT * FROM customers WHERE YEAR(join_date) = 2024 LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
12	18:25:05	SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01' LIMIT 0, 1000	8 row(s) returned	0.016 sec / 0.000 sec
13	18:40:55	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
215 AND join_date < '2025-01-01';
216
217 -- Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
218 -- Q13. Count the total number of orders in the orders table.
219 SELECT COUNT(*) AS total_orders
220 FROM orders;
221
222 -- Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.
223 SELECT SUM(total_amount) AS delivered_revenue
224 FROM orders
225 WHERE status = 'Delivered';
```

The Result Grid shows the output of the query:

delivered_revenue
17191.00

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
12	18:25:05	SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01' LIMIT 0, 1000	8 row(s) returned	0.016 sec / 0.000 sec
13	18:40:55	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
14	18:42:18	SELECT SUM(total_amount) AS delivered_revenue FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec

15. Calculate the average unit_price of products in each category.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
-- Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.
SELECT SUM(total_amount) AS delivered_revenue
FROM orders
WHERE status = 'Delivered';

-- Q15. Calculate the average unit_price of products in each category.
SELECT
    category,
    AVG(unit_price) AS average_price
FROM products
GROUP BY category;
```

The Result Grid shows the output of the second query:

category	average_price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

The Action Output pane shows the execution of three queries:

#	Time	Action	Message	Duration / Fetch
13	18:40:55	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
14	18:42:18	SELECT SUM(total_amount) AS delivered_revenue FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
15	18:43:38	SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
AVG(unit_price) AS average_price
FROM products
GROUP BY category;

-- Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.
SELECT
    status,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

The Result Grid shows the output of the second query:

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Canceled	1	2999.00
Pending	1	1299.00

The Action Output pane shows the execution of three queries:

#	Time	Action	Message	Duration / Fetch
14	18:42:18	SELECT SUM(total_amount) AS delivered_revenue FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
15	18:43:38	SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
16	18:45:42	SELECT status, COUNT(*) AS total_orders, SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC	4 row(s) returned	0.000 sec / 0.000 sec

17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
277 FROM orders;
278 GROUP BY status;
279 ORDER BY total_revenue DESC;
280
281 -- Q17: Find the most expensive (MAX) and cheapest (MIN) product in each category.
282 SELECT
283     category,
284     MAX(unit_price) AS highest_price,
285     MIN(unit_price) AS lowest_price
286 FROM products
287 GROUP BY category;
```

The Result Grid shows the following data:

category	highest_price	lowest_price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

The Action Output panel shows the execution of the query, with the following message:

```
17 18:47:12 SELECT category, MAX(unit_price) AS highest_price, MIN(unit_price) AS lowest_price FROM products... 3 row(s) returned
```

18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
286 FROM products
287 GROUP BY category;
288
289 -- Q18: List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)
290 SELECT
291     category,
292     AVG(unit_price) AS average_price
293 FROM products
294 GROUP BY category
295 HAVING AVG(unit_price) > 2000;
```

The Result Grid shows the following data:

category	average_price
Clothing	2699.000000
Electronics	2224.000000

The Action Output panel shows the execution of the query, with the following message:

```
18 18:49:20 SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING AVG... 2 row(s) returned
```

SECTION – D

19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
-- Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.
SELECT
  o.order_id,
  o.order_date,
  c.first_name,
  c.last_name,
  o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

The Result Grid shows the following data:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4468.00
1004	2024-08-10	Aarav	Sharma	3469.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7468.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehra	6098.00
1010	2024-08-28	Divya	Nair	1598.00

The Output pane shows the execution log:

Time	Action	Message	Duration / Fetch
17 18:47:12	SELECT	category, MAX(unit_price) AS highest_price, MIN(unit_price) AS lowest_price FROM products...	3 row(s) returned 0.000 sec / 0.000 sec
18 18:49:20	SELECT	category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING AV...	2 row(s) returned 0.000 sec / 0.000 sec
19 18:54:29	SELECT	o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o INN...	10 row(s) returned 0.000 sec / 0.000 sec

20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
-- Q20. Using a LEFT JOIN, list all customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  o.order_id,
  o.order_date,
  o.status
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

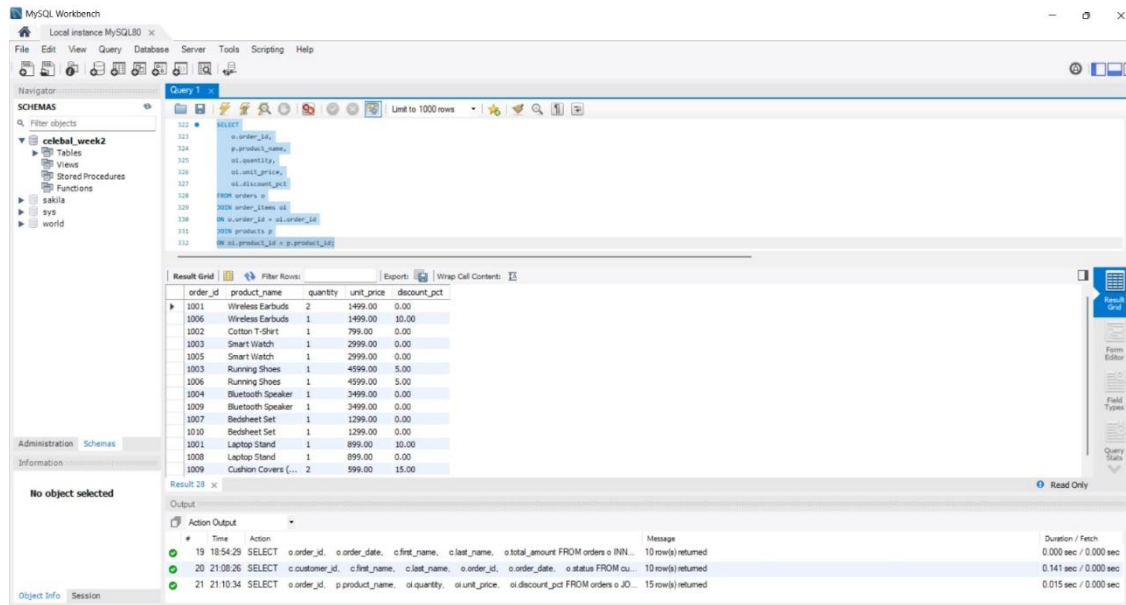
The Result Grid shows the following data:

customer_id	first_name	last_name	order_id	order_date	status
101	Aarav	Sharma	1001	2024-08-01	Delivered
101	Aarav	Sharma	1004	2024-08-10	Delivered
102	Priya	Patel	1002	2024-08-03	Delivered
103	Rohan	Gupta	1003	2024-08-05	Shipped
103	Rohan	Gupta	1008	2024-08-20	Delivered
104	Sneha	Reddy	1005	2024-08-12	Cancelled
105	Vikram	Singh	1006	2024-08-15	Delivered
106	Ananya	Iyer	1007	2024-08-18	Pending
107	Karan	Mehra	1009	2024-08-25	Shipped
108	Divya	Nair	1010	2024-08-28	Delivered

The Output pane shows the execution log:

Time	Action	Message	Duration / Fetch
18 18:49:20	SELECT	category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING AV...	2 row(s) returned 0.000 sec / 0.000 sec
19 18:54:29	SELECT	o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o INN...	10 row(s) returned 0.000 sec / 0.000 sec
20 21:08:26	SELECT	c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.status FROM cu...	10 row(s) returned 0.141 sec / 0.000 sec

21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.



22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

/* LEFT JOIN:

Returns all records from the left table and matching records from the right table.

Example:

customers LEFT JOIN orders

All customers will appear even if they have no orders.

RIGHT JOIN:

Returns all records from the right table and matching records from the left table.

Example:

customers RIGHT JOIN orders

All orders will appear even if customer information is missing.

FULL OUTER JOIN:

Returns all matching and non-matching rows from both tables.

Used when we need complete data from both tables. */

23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

/* Foreign Key Relationships:

1. orders.customer_id
references customers.customer_id

2. order_items.order_id
references orders.order_id

3. order_items.product_id
references products.product_id

If customer_id = 999 does not exist in customers table,
MySQL will throw a Foreign Key Constraint Error
and the record will not be inserted. */

/* The orders table contains a foreign key customer_id
which references customers.customer_id.

If we try to insert an order with customer_id = 999,
and no such customer exists in the customers table,
MySQL rejects the insertion and raises a Foreign Key
Constraint Error.

This ensures referential integrity and prevents
orphan records in the database. */

SECTION – E

24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL query that classifies products into price tiers using a CASE statement. The query is as follows:

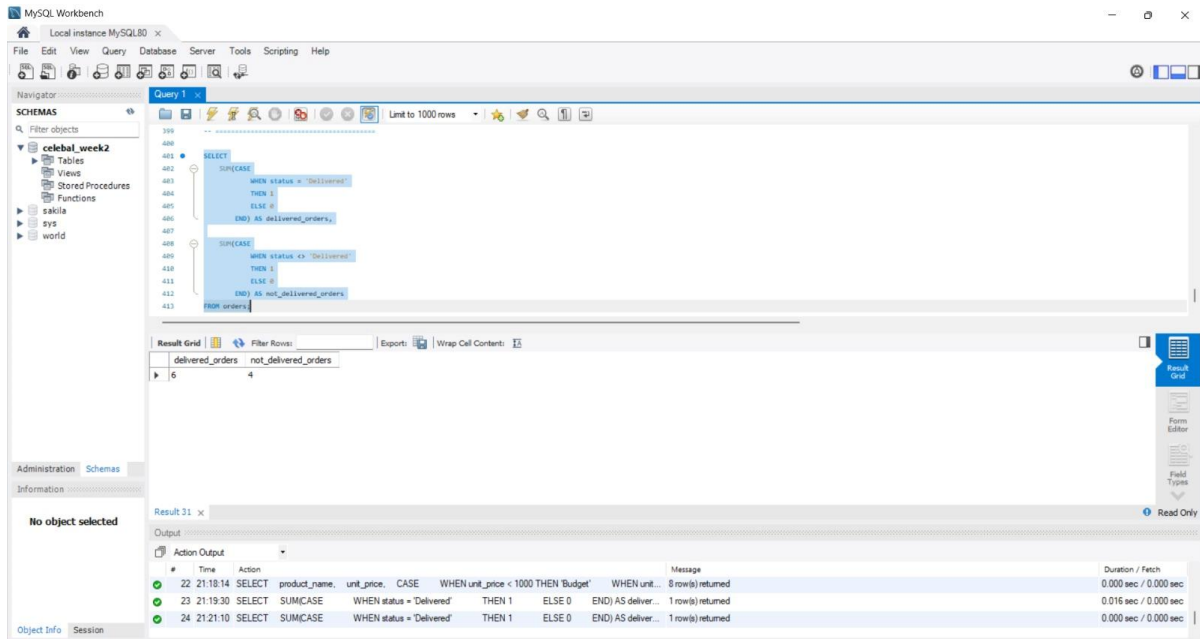
```
SELECT
  product_name,
  unit_price,
  CASE
    WHEN unit_price < 1000 THEN 'Budget'
    WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
    ELSE 'Premium'
  END AS price_tier
FROM products;
```

The 'Result Grid' shows the output of the query, displaying the product name, unit price, and the assigned price tier.

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

The 'Output' tab at the bottom shows the execution log, indicating that the query was executed successfully and returned 8 rows.

25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.



Q26. Explain each letter of ACID:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

Give a real-world example (e.g., bank transfer) showing why each property is important.

/* A – Atomicity

A transaction is completed entirely or not at all.

Example:

If money is deducted from Account A but not credited to Account B due to failure, the entire transaction is rolled back.

C – Consistency

A transaction must keep the database in a valid state.

Example:

After a bank transfer, total money in the system remains unchanged.

I – Isolation

Multiple transactions should not interfere with each other.

Example:

Two users withdrawing money simultaneously should not see inconsistent balances.

D – Durability

Once a transaction is committed, it remains saved even after system crashes.

Example:

Completed bank transfer remains recorded after power failure. */

Q27. Write a SQL transaction that does the following atomically:

- 1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)**
- 2. Insert two order items for that order**
- 3. Update the stock_qty of the purchased products**
- 4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.**

Write the complete BEGIN...COMMIT/ROLLBACK block.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

celebal_week2

Tables

Views

Stored Procedures

Functions

sakila

sys

world

Query 1

Limit to 1000 rows

```
474 UPDATE products
475 SET stock_qty = stock_qty - 1
476 WHERE product_id = 1012;
477
478 COMMIT;
479
480 -- ROLLBACK
481 START TRANSACTION;
482
483 INSERT INTO orders
484 (order_id, customer_id, order_date, status, total_amount)
485 VALUES
486 (1012, 101, CURDATE(), 'Pending', 1000);
487
488 ROLLBACK;
489
490 SELECT *
491 FROM orders
492 WHERE order_id = 1012;
```

Result Grid

order_id	customer_id	order_date	status	total_amount
1012	101	2023-10-27	Pending	1000

Administration Schemas

Information

No object selected

orders 22 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
33	21:25:21	INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (1012, 102, CURDATE(), 'Pending', 1000);	1 row(s) affected	0.000 sec
34	21:25:21	ROLLBACK	0 row(s) affected	0.015 sec
35	21:27:21	SELECT * FROM orders WHERE order_id = 1012 LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

